

Software Architecture and Design Specification

Project: Blood Bank Management System

Version: 1.0

Authors: Team 9

Date: 03-10-2026

Status: Draft

Revision History

Version	Date	Author	Change Summary
1.0	03-10-2026	Team 9	Initial SAD draft

Approvals

Role	Name	Signature / Date
QA Lead	Purav N Khincha	TBD
Requirements & SRS Lead	Prakruti Prasanna Bhat	TBD
Architecture & Design Lead	Pallavi Rajkumar Bubanale	TBD
Project Lead / Product Owner	Ramith Munnann Ravindranath	TBD

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Blood Bank Management System (BBMS). It describes the major software components, their interactions, architectural structure, technology environment, security architecture, and detailed design of the system. The document serves as a reference for implementation, testing, maintenance, and future enhancement of the system.

1.2 Scope

The Blood Bank Management System is a web-based application designed to support blood bank operations. The system manages donor information, donation records, blood inventory, expiry and

stock monitoring, blood availability search, blood requests, blood issue and fulfilment, and reports and history.

The system supports Blood Bank Staff, Hospital Organization Staff, and Administrators. It uses a client-server architecture with a Python and Django backend and a relational database. The system provides role-based access control and supports secure management of blood-related records and transactions.

1.3 Audience

This document is intended for the developers, architecture and design team, testers, project members, faculty evaluators, and other stakeholders involved in the development, testing, maintenance, and evaluation of the Blood Bank Management System.

1.4 Definitions

Term	Definition
BBMS	Blood Bank Management System
SRS	Software Requirements Specification
SAD	Software Architecture and Design Specification
STP	Software Test Plan
FR	Functional Requirement
NFR	Non-Functional Requirement
RBAC	Role-Based Access Control
Blood Inventory	Available blood stock maintained according to blood group and units
Blood Request	Request submitted by an authorized hospital/requesting organization for a required blood group and quantity
Blood Issue	Process of issuing available blood against an approved request
Near-expiry	A blood unit approaching its expiry date within the configured monitoring period
Low Stock	A blood group whose available quantity is below the defined minimum level

2. Document Overview

2.1 How to use this document

This document describes the architecture and design of the Blood Bank Management System. Section 3 describes the system architecture, including the major components, their responsibilities, architecture pattern, technology stack, risks, requirements traceability, and security architecture. Section 4 describes the detailed design, including important system workflows, UML sequence diagrams, API design, error handling, logging, monitoring, and user interface considerations.

This document should be used together with the Software Requirements Specification (SRS) and Software Test Plan (STP) during implementation and testing.

2.2 Related Documents

Document	Description
Software Requirements Specification (SRS)	Defines the functional, non-functional, safety, security, business, and domain requirements of BBMS.
Software Test Plan (STP)	Defines the testing strategy, test cases, test environment, and requirements traceability for BBMS.
Project Guidelines	Defines the required documentation and project deliverables.

3. Architecture

3.1 Goals & Constraints

Goals

- Maintain accurate and up-to-date blood inventory for different blood groups.
- Support management of donors, donations, blood requests, blood issues, and reports.
- Ensure that expired blood units are not treated as available for issue.
- Prevent inventory quantities from becoming negative.
- Enforce role-based access control so users can access only the functions permitted for their roles.
- Maintain consistency between blood inventory and related transactions.
- Provide a modular design that supports testing, maintenance, and future enhancement.
- Provide a web-based system for managing blood bank operations.

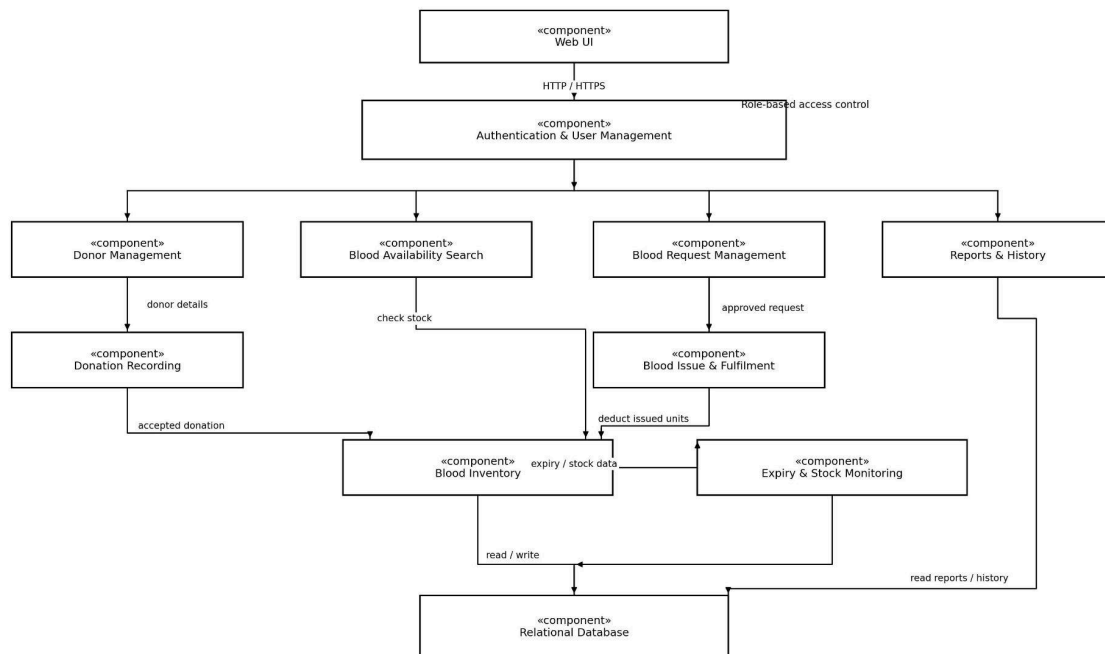
Constraints

- The system is implemented as a web-based application.
- The proposed implementation uses Python and Django with a relational database.
- The system follows a client-server architecture.
- Communication between the client and server uses HTTP/HTTPS, with HTTPS required for deployment.
- Access to system functions is controlled using role-based access control.
- Version 1.0 does not include integration with external hospital systems.
- Medical screening and physical laboratory testing of blood are outside the software system boundary.
- The system depends on a web browser, application server, and database server for operation.

3.2 Stakeholders & Concerns

Stakeholder	Concerns
Blood Bank Staff	Donor and donation management, accurate blood inventory, expiry and low-stock monitoring, processing blood requests, and issuing blood.
Hospital Organization Staff	Searching blood availability, submitting blood requests, and tracking request status.
Administrator	User management, access control, system records, and reports.
Developers	Modular architecture, maintainability, database consistency, and ease of testing.
Testers / QA Team	Testability of system components, requirement traceability, functional correctness, security, reliability, and performance.
Project Team / Faculty	Compliance with requirements, system quality, security, maintainability, and successful project evaluation.

3.3 Component (UML) Diagram



3.4 Component Descriptions

Component	Description
Web UI	Provides the browser-based interface through which users access BBMS functions such as donor management, inventory, blood search, requests, issue processing, and reports. The Web UI communicates with the application/business logic layer through the defined REST/API endpoints.
Authentication & User Management	Authenticates users and manages user roles and permissions for Blood Bank Staff, Hospital Organization Staff, and Administrators.
Donor Management	Manages donor information, including donor registration, searching, updating donor details, and viewing donation history.

Donation Recording	Records donation details such as donation date, blood group, units, and donation status for registered donors.
Blood Inventory	Maintains the available blood stock for each blood group. Accepted donations increase inventory, while successful blood issue transactions decrease inventory.
Expiry & Stock Monitoring	Monitors blood units for expiry and near-expiry conditions and identifies blood groups whose available stock is below the defined minimum level.
Blood Availability Search	Allows authorized users to search for available blood by blood group and view the available quantity. Expired units are excluded from availability.
Blood Request Management	Allows authorized hospital staff to submit blood requests and allows blood bank staff to review and manage request status.
Blood Issue & Fulfilment	Handles the fulfilment of approved blood requests and issues blood after checking sufficient inventory. Successful issue transactions update inventory and create an issue record.
Reports & History	Provides historical information and reports related to donations, requests, blood issues, and inventory.
Relational Database	Provides persistent storage for user, donor, donation, inventory, request, issue, and report-related data and maintains relationships and data consistency.

3.5 Chosen Architecture Pattern and Rationale

Chosen Architecture Pattern

The Blood Bank Management System follows a layered architecture within a client-server model. The system is organized into separate layers for presentation, application/business logic, and data management. This separation allows each layer to focus on a specific responsibility and supports modular development, testing, maintenance, and future enhancement.

Layers

Layer	Responsibility
Presentation Layer	Provides the web-based user interface through which Blood Bank Staff, Hospital Organization Staff, and Administrators interact with the system.
Application / Business Logic Layer	Implements authentication, donor and donation management, inventory management, expiry and stock monitoring, blood search, request processing, issue and fulfilment, and reporting logic.
Data Management Layer	Handles persistent storage and retrieval of users, donors, donations, inventory, requests, issue records, and related data using the relational database.

Each major functional area is organized as a separate Django application to support modularity, maintainability, and separation of responsibilities.

Rationale

Layered architecture is selected because it provides clear separation of responsibilities between the user interface, application logic, and data management. It supports independent testing of system modules and makes it easier to maintain and modify individual components. It also fits the web-based client-server structure and the proposed Python and Django implementation described in the SRS.

3.6 Technology Stack & Data Stores

Technology Stack

Technology	Purpose
Python	Backend programming language
Django	Web application framework and application logic
HTML/CSS	Web interface
JavaScript	Client-side interaction
PostgreSQL	Relational database for persistent data storage

Gunicorn	Application server
Nginx	Web server and reverse proxy
HTTPS/TLS	Secure communication
GitHub	Source code and version control

Data Stores

The system uses a relational database to store and manage:

- User accounts and roles
- Donor details
- Donation records
- Blood inventory
- Blood request records
- Blood issue records
- Historical and reporting data

Database Choice

PostgreSQL is selected as the relational database for the Blood Bank Management System. It provides persistent storage for system data and supports relationships and consistency between inventory and transaction records.

3.7 Risks & Mitigations

Risk	Mitigations
Incorrect inventory updates	Validate inventory changes and ensure that successful transactions update stock consistently.
Expired blood being issued	Monitor expiry dates and exclude expired units from available inventory.
Inventory becoming negative	Verify sufficient stock before issuing blood and prevent transactions that exceed available quantity.
Unauthorized access to system functions	Use authentication and role-based access control for protected functions.
Database or transaction failure causing inconsistent data	Use transactional operations so failed operations do not partially update inventory.
Delays in development or integration between modules	Use modular components, unit testing, and integration testing during development.

Changes to requirements affecting test cases and design	Maintain requirements traceability and update affected design and test documentation when requirements change.
---	--

3.8 Traceability to Requirements

Requirement ID	Requirement	Architecture/Component Reference	Design Reference
FR-1.1–FR-1.7	Donor Registration	Donor Management	Donor management workflow
FR-2.1–FR-2.7	Donation Recording	Donation Recording	Donation recording workflow
FR-3.1–FR-3.5	Blood Inventory Management	Blood Inventory	Inventory update and validation
FR-4.1–FR-4.5	Expiry & Stock Monitoring	Expiry & Stock Monitoring	Expiry and low-stock monitoring
FR-5.1–FR-5.4	Blood Availability Search	Blood Availability Search	Blood availability search
FR-6.1–FR-6.7	Blood Request Management	Blood Request Management	Request submission and processing
FR-7.1–FR-7.7	Blood Issue & Fulfilment	Blood Issue & Fulfilment	Blood issue and inventory update
FR-8.1–FR-8.5	Reports & History	Reports & History	Reporting and history retrieval
NFR-01	Donor search, blood availability search and inventory view respond within 3 seconds	Blood Availability Search, Blood Inventory	API and database operations
NFR-02	Submission of donations, requests and issues is processed within 3 seconds	Application / Business Logic Layer	API and transaction processing
NFR-03	Reports display within 5 seconds	Reports & History	Report retrieval
NFR-04	Inventory changes are reflected immediately	Blood Inventory, Relational Database	Inventory transaction processing

NFR-05	Trained staff can record a donation within 2 minutes	Web UI, Donation Recording	Donation recording workflow
NFR-06	Failed transactions leave inventory unchanged	Blood Inventory, Relational Database	Transaction management / error handling
NFR-07	Functional areas are implemented as separate Django apps	Layered Architecture, System Components	Modular component design
NFR-08	Application works across supported browsers and operating systems	Web UI	Web-based client-server architecture
NFR-09	Every functional requirement has at least one test case	All System Components	Requirements traceability and test design
SAFE-01	Expired blood units shall not be shown as available for issue	Expiry & Stock Monitoring, Blood Issue & Fulfilment	Blood issue validation
SAFE-02	Blood shall be issued only against an approved blood request	Blood Request Management, Blood Issue & Fulfilment	Request approval and issue validation
SAFE-03	Blood shall not be issued when sufficient stock is unavailable	Blood Inventory, Blood Issue & Fulfilment	Inventory validation
SAFE-04	Confirmation is required before completing a blood-issue operation that changes inventory	Blood Issue & Fulfilment	Blood issue confirmation
SAFE-05	Medical screening and final suitability of donated blood remain outside the software	System Boundary	Out-of-scope operation
SEC-01	Users shall authenticate before accessing protected functions	Authentication & User Management	Authentication design
SEC-02	The system shall provide role-based	Authentication & User Management	RBAC design

	access to system functions		
SEC-03	Only authorized blood-bank staff shall modify inventory or record blood issuance	Authentication & User Management, Blood Inventory, Blood Issue & Fulfilment	Authorization checks
SEC-04	Administrative functions shall be restricted to administrator accounts	Authentication & User Management	Admin authorization
SEC-05	Passwords shall not be displayed in plain text during entry	Authentication & User Management	Password handling
SEC-06	Communication between browser and deployed application should use HTTPS	Web UI, Application / Business Logic Layer	Secure communication
SO-01	Confidentiality	Authentication & User Management, Web UI	Authentication and access control
SO-02	Integrity	Blood Inventory, Blood Request Management, Blood Issue & Fulfilment, Relational Database	Authorization, validation and transaction management
SO-03	Access Control	Authentication & User Management	RBAC and authorization

The architecture and design are further validated through the Software Test Plan. The test cases are mapped to the corresponding functional and non-functional requirements to ensure complete verification of the implemented system.

3.9 Security Architecture

The security architecture of the Blood Bank Management System is designed to protect user accounts, blood-related records, inventory data, and system operations. Security is implemented through authentication, role-based access control, secure communication, authorization checks, and data validation.

Security objectives:

- SO-01 – Confidentiality: Prevent unauthorized access to donor information, user information, blood requests, and other protected system records.

- SO-02 – Integrity: Protect donation, inventory, request, and blood-issue records from unauthorized modification.
- SO-03 – Access Control: Ensure that users can access only the functions permitted for their assigned role.

Security mechanisms:

Security Mechanism	Description
Authentication	Users must authenticate before accessing protected system functions.
Role-Based Access Control	Access permissions are assigned according to user roles such as Blood Bank Staff, Hospital Organization Staff, and Administrator.
Authorization	The system checks whether the authenticated user is authorized to perform a requested operation.
Password Protection	Passwords are protected and not displayed as plain text during login.
HTTPS/TLS	HTTPS is used to protect communication between the client and server during deployment.
Input Validation	User inputs are validated before being processed or stored in the database.
Inventory Validation	The system verifies available stock before issuing blood and prevents negative inventory.
Transaction Management	Database transactions help prevent partial updates when an operation fails.
Audit / History Records	Important donation, request, issue, and inventory-related activities are maintained for history and reporting.

Role-based access:

Role	Main Access
Blood Bank Staff	Manage donors and donations, view and manage inventory, monitor expiry and stock levels, process blood requests, issue blood, and view reports.

Hospital Organization Staff	Search blood availability, submit blood requests, and view request status.
Administrator	Manage users, roles, access control, and administrative records.

Security architecture flow:

User → Authentication → Role & Permission Check → Authorized Component → Business Logic Validation → Relational Database

Unauthorized requests are rejected before the protected operation is performed. Inventory-related operations additionally validate stock availability, request status, and expiry conditions before modifying inventory.

4. Design

4.1 Design Overview

The detailed design of the Blood Bank Management System defines how the architectural components interact to implement the functional and non-functional requirements. The design follows the layered architecture described in Section 3.5.

The Presentation Layer provides the web interface through which users interact with the system. Requests from the user interface are handled by the Application / Business Logic Layer, which performs authentication, authorization, validation, processing, and business operations. The Data Management Layer communicates with the relational database to store and retrieve system information.

The major design areas include:

- User authentication and role-based access control.
- Donor and donation management.
- Blood inventory management and validation.
- Expiry and low-stock monitoring.
- Blood availability search.
- Blood request processing.
- Blood issue and fulfillment.
- Reports and historical record retrieval.
- Error handling, logging, and monitoring.
- Secure communication and data protection.

The design ensures that inventory-related operations are validated before modifying stock. Accepted donations increase inventory, while successful blood issues decrease inventory. Expired blood is excluded from available stock, and transactions that would result in negative inventory are rejected.

4.2 UML Sequence Diagrams

4.2.1 Blood Donation and Inventory Update

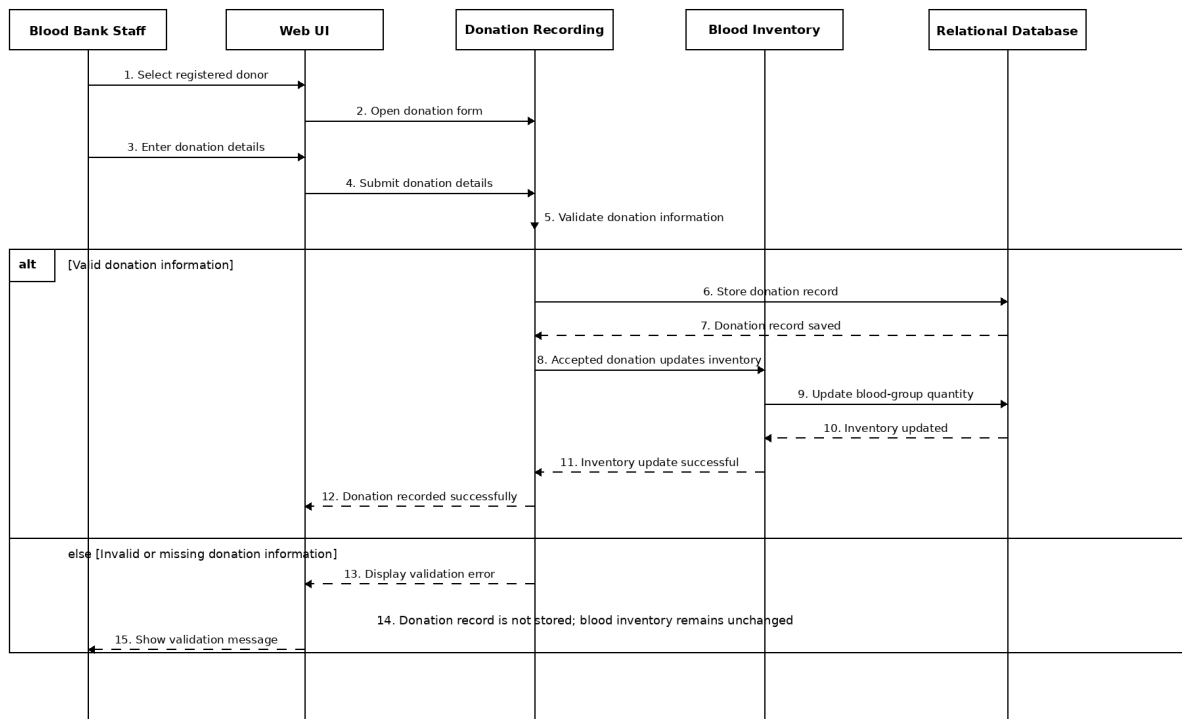


Figure 4.1: Sequence diagram for Blood Donation and Inventory Update

4.2.2 Blood Request and Issue Fulfilment

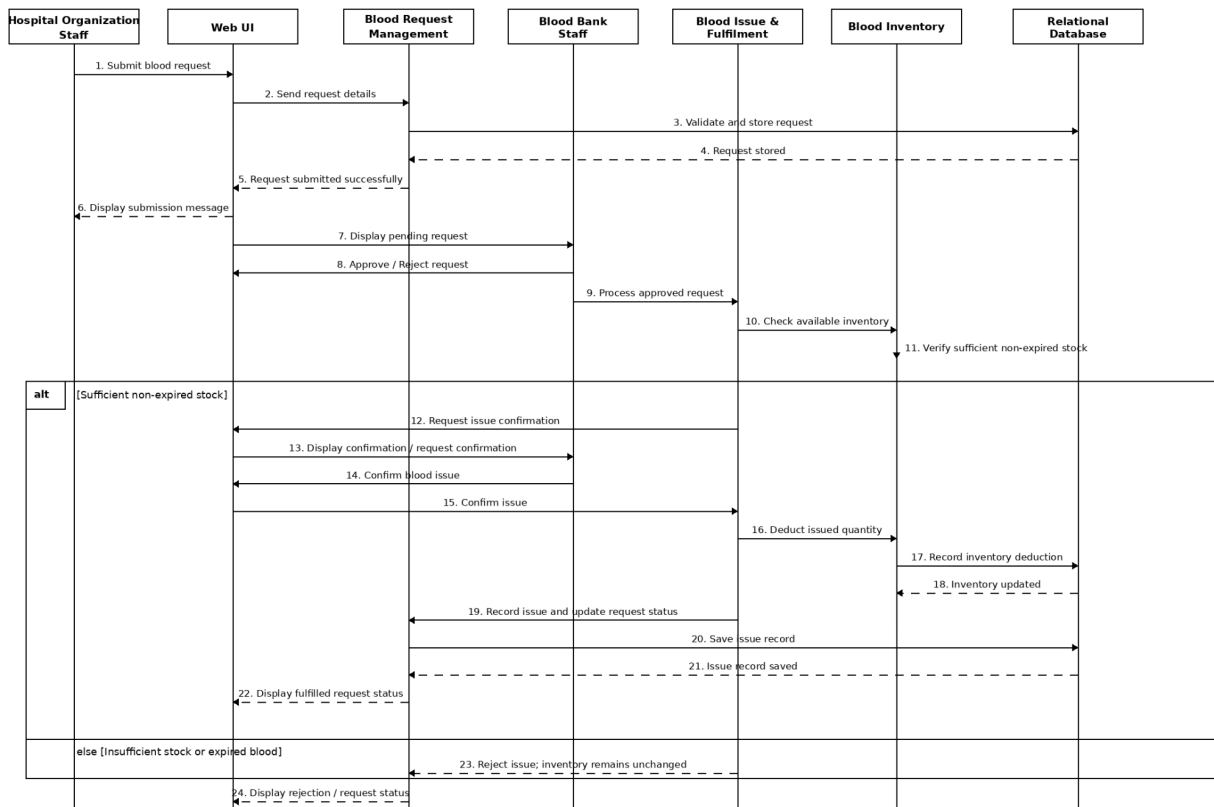


Figure 4.2: Sequence diagram for Blood Request and Issue Fulfilment

4.3 API Design

The Blood Bank Management System provides REST/API endpoints for communication between the Web UI and the application/business logic layer. The APIs support authentication, donor management, donation recording, blood inventory, blood availability search, blood requests, blood issue and fulfilment, and reports.

API EndPoint	Method	Purpose	Access
/api/auth/login	POST	Authenticate a user and create a session	All users
/api/auth/logout	POST	Log out the current user	Authenticated users
/api/donors	GET	Retrieve donor information	Blood Bank Staff
/api/donors	POST	Register a new donor	Blood Bank Staff
/api/donors/{id}	GET	View donor details and history	Blood Bank Staff

/api/donors/{id}	PUT	Update donor information	Blood Bank Staff
/api/donations	POST	Record a blood donation	Blood Bank Staff
/api/inventory	GET	View current blood inventory	Blood Bank Staff
/api/inventory/{blood_group}	GET	View availability for a blood group	Authorized users
/api/monitoring	GET	Retrieve expiry and low-stock information	Blood Bank Staff
/api/requests	POST	Submit a blood request	Hospital Organization Staff
/api/requests	GET	View blood requests and their status	Authorized users
/api/requests/{id}/approve	POST	Approve a blood request	Blood Bank Staff
/api/requests/{id}/reject	POST	Reject a blood request	Blood Bank Staff
/api/issues	POST	Issue blood against an approved request	Blood Bank Staff
/api/reports	GET	Retrieve reports and historical information	Authorized users

API design principles:

- Authentication is required for protected endpoints.
- Role-based authorization is applied according to the user's role.
- Input data is validated before processing.
- Successful blood issue operations update inventory consistently.
- The API prevents issuing more blood than the available quantity.
- Expired blood is excluded from availability and issue operations.
- Appropriate HTTP status codes and error messages are returned for invalid or unauthorized requests.
- HTTPS is used during deployment to protect API communication.

4.4 Error Handling, Logging & Monitoring

The Blood Bank Management System uses validation, error handling, logging, and monitoring mechanisms to maintain system reliability, data consistency, and security.

4.4.1 Error Handling

The system validates user input and business rules before performing operations. Errors are handled without exposing sensitive system information to users.

Error Condition	Handling
Invalid or missing input	Display a validation message and request correction.
Unauthorized access	Reject the request and display an access-denied message.
Invalid login credentials	Reject authentication and display an appropriate error message.
Blood request not approved	Prevent blood issue and display the request status.
Insufficient blood stock	Reject the issue operation and keep inventory unchanged.
Expired blood	Exclude expired blood from availability and issue operations.
Invalid donor or donation data	Reject the operation and display a validation message.
Database or transaction failure	Roll back the affected transaction and prevent partial updates.
Requested record not found	Return an appropriate not-found response.

4.4.2 Logging

The system maintains logs for important system and transaction events. Logging supports troubleshooting, security monitoring, and system maintenance.

Important events include:

- User authentication and logout events.
- Unauthorized access attempts.

- Donor and donation operations.
- Blood inventory updates.
- Blood request approval or rejection.
- Blood issue and fulfilment operations.
- Validation failures and system errors.
- Database or transaction failures.

Logs should not contain passwords or other sensitive authentication information.

4.4.3 Monitoring

The system monitors important operational conditions, including:

- Blood inventory levels.
- Near-expiry and expired blood units.
- Low-stock blood groups.
- Failed or rejected transactions.
- Application and database errors.
- Unauthorized access attempts.

Monitoring information can be used by authorized Blood Bank Staff and Administrators to identify operational or security issues and take appropriate action.

4.5 UX Design

The Blood Bank Management System provides a simple and consistent web-based user interface for different user roles. The interface is designed to allow users to access only the functions permitted for their roles.

4.5.1 User Interface Structure

The main interface provides navigation to the functions available to the authenticated user.

User Role	Main Functions
Blood Bank Staff	Donor Management, Donation Recording, Blood Inventory, Expiry & Stock Monitoring, Blood Requests, Blood Issue & Fulfilment, Reports & History
Hospital Organization Staff	Blood Availability Search, Blood Request Management, Request Status
Administrator	User Management, Role Management, Access

	Control, Administrative Records
--	---------------------------------

4.5.2 UX Design Principles

- Provide clear navigation between major system functions.
- Display information using readable tables, forms, and status indicators.
- Provide clear confirmation messages after successful operations.
- Display meaningful error and validation messages when an operation fails.
- Restrict navigation and actions according to the user's role.
- Clearly display blood group, available quantity, request status, and expiry-related information.
- Require confirmation before operations that modify blood inventory.
- Use consistent form layouts, buttons, labels, and messages throughout the system.
- Provide responsive web pages that can be accessed using modern web browsers.

4.6 Open Issues & Next Steps

The following items remain open for implementation, integration, testing, and future enhancement of the Blood Bank Management System.

- Complete implementation of all architecture and design components defined in this document.
- Integrate the Web UI with the REST/API endpoints and application logic.
- Finalize and configure the PostgreSQL database for deployment.
- Complete integration testing between Donation Recording, Blood Inventory, Blood Request Management, and Blood Issue & Fulfilment.
- Validate role-based access control for all protected functions.
- Perform security, reliability, and performance testing according to the Software Test Plan.
- Resolve defects identified during unit, integration, system, and acceptance testing.
- Maintain traceability between requirements, architecture, design, implementation, and test cases.
- Consider future enhancements such as external hospital system integration and additional user-facing features as defined in the SRS future scope.

5. Appendices

5.1 Glossary

Term	Definition
BBMS	Blood Bank Management System
SRS	Software Requirements Specification

SAD	Software Architecture and Design Specification
STP	Software Test Plan
FR	Functional Requirement
NFR	Non-Functional Requirement
RBAC	Role-Based Access Control
API	Application Programming Interface
REST	Representational State Transfer
HTTPS	Hypertext Transfer Protocol Secure
TLS	Transport Layer Security
Blood Inventory	Available blood stock maintained according to blood group and quantity
Blood Request	Request submitted by an authorized hospital organization for required blood
Blood Issue	Process of issuing blood against an approved request
Near-expiry	Blood approaching its expiry date within the configured monitoring period
Low Stock	Blood group whose available quantity is below the defined minimum level
PostgreSQL	Relational database management system used for persistent data storage
Django	Python web application framework used for developing the system

5.2 References

1. Software Requirements Specification (SRS) – Blood Bank Management System, Team 9, Version 1.0.
2. Software Test Plan (STP) – Blood Bank Management System, Team 9, Version 1.0.
3. Software Architecture and Design Specification (SAD) Guidelines – Software Engineering Mini Project, Part-1.

4. Django Documentation – Python web application framework documentation.
5. PostgreSQL Documentation – Relational database management system documentation.

5.3 Tools

Tool	Purpose
Python	Backend application development
Django	Web application framework
PostgreSQL	Relational database management
HTML/CSS	Web interface development
JavaScript	Client-side interaction
Gunicorn	Application server
Nginx	Web server and reverse proxy
GitHub	Source code management and version control
draw.io	Architecture and UML diagram creation
Google Docs	Documentation and report preparation
Web Browser	Accessing and testing the web application